Claude Code: An Information-Theoretic Deconstruction of Large Language Model Agent Architecture

WallXiaoming

https://github.com/WallXiaoming

June 2026

Abstract

As large language models evolve from static conversational tools into autonomous agent systems capable of interacting with software environments, the software engineering paradigm is undergoing a fundamental restructuring. This paper presents a deep deconstruction of Anthropic's Claude Code—a state-of-the-art coding agent—through the lens of information theory. We propose a novel theoretical framework: modeling LLM agent systems as multi-layer entropy reduction pipelines. Through analysis of leaked source code and the latest 2026 benchmark data, we demonstrate that Claude Code's architecture is fundamentally a `while(true)` loop that iteratively reduces task uncertainty by injecting deterministic tool execution results. We apply a Fano-style accuracy upper bound to prove that single-pass LLM generation inevitably faces an *accuracy cliff* when task information demand exceeds model capacity, and show how tool calling—an emergent zero-shot capability—mathematically circumvents this limit through capacity-aware task decomposition. We further analyze systemic vulnerabilities including *termination poisoning* (LoopTrap) and *silent failures*, formalized through the Intelligence Entropy Principle $S(t) = S_0 e^{\lambda t}$. Finally, we propose the **Real-Time Entropy Evaluation System** (RTEES), a middleware architecture that replaces the model's subjective completion judgment with objective, entropy-threshold-based termination criteria. This work provides a structured, quantifiable framework for understanding and building next-generation reliable multi-agent systems.

1 Introduction

In the early stages of artificial intelligence research, large language models were primarily regarded as statistical text completion engines. However, as model parameter scales expanded and reinforcement learning fine-tuning matured, models gradually developed the ability to perceive and manipulate their environments, heralding the age of LLM-based agents.

Claude Code, released by Anthropic, represents the forefront of this evolution. It is an autonomous coding agent capable of reading codebases, writing and editing files, executing shell commands, and performing multi-step software engineering tasks. Through a systematic reverse engineering of its leaked source code[1] and analysis of the latest 2026 benchmarks, we observe that Claude Code's architecture exhibits remarkable stability and paradigm convergence.

[1] Leaked version dated 2026-03-31.

1.1 Core Architecture Formula

The operational essence of Claude Code can be summarized by a minimal system equation:

$$\text{Claude Code} = \underbrace{\text{Context}}_{\text{reduces input entropy}} + \underbrace{\text{Tools}}_{\text{reduces execution entropy}} + \underbrace{\texttt{while(true) loop}}_{\text{continuous reduction}} \tag{1}$$

This formulation aligns precisely with founder Boris Cherny's own description: "You have a model, you give it tools, you give it some context and a task, and it uses the tools to accomplish the task." Despite 16 months of intensive iteration—introducing permission systems, `CLAUDE.md`, AgentTool, Skill, MCP, Hooks, Compact, Plan Mode, and Advisor—every enhancement has been an optimization around this core loop, not a restructuring of it.

1.2 The Core Execution Loop

From the source code analysis (`src/query.ts`), the agent's lifecycle is encapsulated within a single iterative block that continuously communicates with the model API:

```
while (true) {
  const response = await callModel({ system, tools, messages })

  if (response.any(block => block.type === "tool_use")) {
    for (const toolCall of response.toolCalls) {
      const result = executeTool(toolCall)
      messages.push(result)
    }
    continue
  } else {
    displayToUser(response.text)
    break
  }
}
```

In each iteration, the system sends the system prompt, collected context, conversation history, and tool definitions to the LLM. If the model determines that it lacks sufficient information to produce the final output, it returns tool call blocks. The system executes these tools and injects the results as new deterministic evidence into the conversation history, triggering another iteration. When the model produces no tool calls, the system presumes task completion and exits.

1.3 Model Context Protocol (MCP)

The success of this architecture is significantly enabled by the Model Context Protocol (MCP)[4, 18], an open standard introduced by Anthropic in late 2024. Inspired by the Language Server Protocol (LSP), MCP employs a lightweight client-server model with JSON-RPC 2.0 for standardized information exchange[4]. In Claude Code, MCP serves as a universal bus connecting the model's internal representations to external environments, providing three core primitives: `Resources` (read-only context), `Tools` (executable functions), and `Prompts` (structured task templates) [11]. This standardized capability exposure mechanism enables models to dynamically acquire external information and modify environment state without retraining, laying the infrastructure foundation for complex task execution.

2 Information-Theoretic Entropy Reduction Pipeline

To deeply understand why the loop architecture solves complex software engineering problems, we adopt information theory as the foundational analytical framework. Under this lens, every system operation and every tool call is inherently a quantified *entropy reduction* operation.

2.1 Shannon Entropy and Task Uncertainty

Claude Shannon's entropy measures the uncertainty of a system or message:

$$H(X) = -\sum_{i=1}^{n} p(x_i) \log_2 p(x_i) \tag{2}$$

In the context of AI interaction, entropy represents the system's uncertainty about the user's true intent, the current project state, and the correct implementation path. Critically, the same natural language input produces vastly different initial entropy values depending on the receiver's background knowledge. An ambiguous instruction such as "complete the production deployment" represents extremely high entropy for a model with no project context—potentially thousands of equally likely solution paths. For a model that has already read the project's architecture documentation and testing conventions, the solution space collapses dramatically.

2.2 Entropy Reduction Pipeline Architecture

Following this logic, Claude Code's architecture can be abstracted as a multi-layer entropy reduction pipeline:

Stage I — Pre-context Collection. The system executes a first round of entropy reduction before the model even begins reasoning: it queries `git status/log` to determine repository state, loads `CLAUDE.md` for project conventions, reads `memdir/` for cross-session persistent memory, and collects environment information. This injects deterministic objective information, compressing the initial task entropy from approximately 3–5 bits down to 1.5–3 bits.

Stage II — Iterative Loop Reduction. In each loop iteration, the model actively acquires external feedback by issuing tool calls. Each successful tool execution and result return is analogous to injecting new observational evidence into a Bayesian network. The model continuously updates its posterior distribution, progressively reducing the conditional entropy $H(\text{output} \mid \text{knowledge}, \text{history})$. When the model's internal probability computation suggests that the remaining information suffices for a unique correct solution, it stops calling tools, at which point the system entropy approaches zero.

2.3 Component Entropy Roles

Under this information-theoretic perspective, each system component assumes a well-defined mathematical role, as summarized in Table 1.

The Compact module operates under information bottleneck theory, striving to retain the highest mutual-information content within a severely limited token budget, ensuring the reduction process does not regress due to memory overflow. The Advisor mechanism introduces external intelligence at high-entropy forks where existing tools cannot resolve ambiguity.

Table 1: System components mapped to their information-theoretic roles.

Component	Information-Theoretic Role
System Prompt	Prior distribution injection: defines model capability boundary
CLAUDE.md	Conditional entropy $H(\text{output} \mid \text{knowledge})$ reduction
memdir/	Cross-session persistence prior
Tool Schema (Zod)	Output space truncation: illegal parameter probability $\to 0$
Tool Execution	Converts model uncertainty into deterministic fact
Compact	Information bottleneck: retains max mutual-info under token budget
Permission	Human arbitration at critical decision points
Skill	Methodology injection: reusable reduction strategy template
Agent	Independent entropy reduction sub-loop (isolated context)
Hook	External deterministic signal injection
Plan Mode	Explicit reduction path design before execution
Advisor	Stronger model intervention at high-entropy forks

3 The Information-Theoretic Limit of Single-Pass Reasoning

A fundamental question arises: why must agent systems rely on tool calls and multi-step loops? Why cannot simply scaling model parameters achieve perfect single-pass solutions? Recent information-theoretic research provides rigorous mathematical answers.

3.1 The Fano-Style Accuracy Upper Bound

Researchers have established a Fano-style accuracy upper bound for single-pass LLM reasoning[5]:

$$P_{\text{correct}} \leq 1 - \frac{H(X \mid Q, C)}{\log |\mathcal{X}|} \tag{3}$$

Given a complex task with ground-truth answer X, natural language query Q, and context C, the task information demand is $H(X \mid Q, C)$—the conditional entropy of the true answer given the available input. The model's single-pass output capacity is bounded by its output distribution's Shannon entropy. The Fano-style bound mathematically states that the maximum achievable accuracy is implicitly bounded by the gap between task information demand and model output capacity[5].

3.2 The Accuracy Cliff

This theory predicts a practically devastating phenomenon: the *accuracy cliff* [5]. Because any single-pass model has finite output capacity, when task complexity increases such that the inherent information demand exceeds the model's capacity ceiling, perfect accuracy becomes mathematically impossible. The model's performance does not degrade smoothly with difficulty but rather collapses catastrophically, exhibiting a double-exponential deterioration[5]. This fundamentally explains why even the largest frontier models still suffer severe hallucinations and logic breaks when facing real-world software engineering tasks that require cross-file navigation and integration of loosely coupled evidence.

3.3 Tool Calling as Capacity-Aware Decomposition

Given this dual physical and mathematical limit, Claude Code's tool-calling loop mechanism becomes an architectural necessity. The essence of tool calling is *capacity-aware task decom-*

position[5]. In each iteration, rather than attempting to produce the entire solution in one pass, the model outputs a highly focused tool call with minimal information demand—far below the model's output capacity ceiling, ensuring high per-step reliability. By persisting intermediate states to external files or memory, the system effectively resets the error accumulation process, bypassing the single-pass accuracy cliff.

Furthermore, the model's ability to call tools is not a specially fine-tuned skill but an emergent zero-shot capability. The model naturally translates tool names, descriptions, and input schemas from the API definition into concrete call instructions. This demonstrates that tool calling transforms abstract "action selection" into "natural language generation with strict syntactic and semantic constraints"—a direct mapping of pre-trained pattern-matching abilities into structured space.

3.4 Pro versus Flash: The Bayesian Prior Advantage

Table 2 compares flagship (Pro) and lightweight (Flash) model performance across dimensions.

Table 2: Flagship versus lightweight model comparison.

Dimension	**Pro**	**Flash**	**Root Cause**
HumanEval (function completion)	$\sim 99\%$	$\sim 97\%$	Minimal (deterministic tasks saturated)
Multi-step reasoning (GPQA Diamond)	94%	74%	Large (ambiguous judgment needs capacity)
Error recovery	Strong	Weak	Parameter count $\rightarrow$ strategy richness
Long-document cross-section reasoning	89%	79%	Attention maintenance gap

On highly deterministic tasks like single-function completion, the gap between Pro and Flash approaches zero (both reaching 95–99% accuracy)[2]. The training signal is so clear and task information demand so low that even small models have sufficient capacity. However, on tasks involving ambiguous signals, multi-step abstract reasoning, or sustained attention over long contexts, the gap remains substantial[1, 16].

The flagship model's massive parameter space essentially purchases a "superior Bayesian prior for high-entropy environments." This yields a key engineering corollary:

$$\text{Flash} + \text{good decomposition} + \text{clear methodology} \approx \text{Pro effectiveness} \tag{4}$$

Anthropic's 2026 Advisor strategy data confirms this: Haiku 4.5 + Opus Advisor achieves 41.2% task success (versus 19.7% baseline), at only 15% of Sonnet's cost.

4 Benchmark Hierarchy and Model Landscape

To precisely measure agent capabilities across information complexity levels, the research community has established a tiered benchmark system that directly reflects model performance on deterministic versus non-deterministic tasks.

4.1 Benchmark Tier Structure

Table 3 summarizes the five-tier evaluation hierarchy [2, 13].

At L1, most frontier models have reached saturation[16]. L2 (SWE-bench Pro) requires models to navigate real open-source repositories with hundreds of files, locate bugs, and generate cross-file patches[12]. L3 (Terminal-Bench, OSWorld) evaluates tool chaining, observation-correction cycles, and multi-step script execution[3]. L4 (SWE-Marathon) tests hours-long sustained task execution where context drift and memory decay become critical. L5 lacks objective machine-verifiable standards, yielding unstable results.

Table 3: Coding agent benchmark hierarchy.

Tier	Representative Benchmarks	Capability
L1 Code Generation	HumanEval / MBPP	Function-level completion (saturated)
L2 Software Eng.	SWE-bench Pro / FrontierSWE	Bug repair, cross-file edits
L3 Agent Capability	Terminal-Bench / MCP-Atlas	Terminal ops, tool invocation
L4 Long-horizon	SWE-Marathon	Hours-long end-to-end development
L5 Aesthetic/Design	Design Arena / Code Arena	Frontend visual output quality

4.2 2026 Model Ecosystem

The 2026 LLM landscape reveals distinct specialization patterns[1–3, 12]. Claude Fable 5 leads SWE-bench Pro at 80.3% resolution rate[12]. Opus 4.8, while scoring slightly lower on raw coding metrics, remains the gold standard for long-sequence tasks requiring sustained logical coherence due to its exceptionally low hallucination rate[2]. GPT-5.5 and GPT-5.4 achieve 75.1% on Terminal-Bench, excelling at dynamic environment interaction[3]. Gemini 3.1 Pro leverages its massive context window and perfect AIME 2025 math score for data-intensive tasks[2]. GLM-5.2 and DeepSeek V4 Pro dominate cost-sensitive production deployments with sub-$1 per million output tokens[1].

4.3 The Verifiability Theorem

A core theorem emerges from cross-benchmark analysis: **model capability improvement is strictly proportional to task verifiability**.

$$\Delta\text{Performance} \propto \text{Task Verifiability} \tag{5}$$

For deterministic tasks with absolute ground truth (mathematics, test-backed code), RL algorithms capture clear, low-entropy reward signals for large-scale parameter updates. For open-ended tasks relying on subjective human feedback, training signals are inherently noisy and high-entropy, severely limiting optimization efficiency. This explains why even the most capable models struggle with open-ended innovation tasks.

5 Systemic Vulnerabilities: Control Flow Hijacking and Silent Failures

Despite the benchmark success of the loop paradigm, two fundamental architectural flaws lurk beneath the surface. These are not specific code bugs but systemic weaknesses rooted in using non-deterministic language models as core controllers.

5.1 Termination Poisoning (LoopTrap)

The first critical flaw is *termination poisoning*. In the current architecture, the sole criterion for breaking the `while(true)` loop is whether the model stops producing tool calls. This delegates the high-entropy judgment of "is the task objectively complete" entirely to the model's internal floating-point confidence threshold—a design that is inherently fragile without external objective state assertions.

The LoopTrap attack framework, introduced in 2026, exposes this vulnerability systematically[9]. Since agents must continuously read external web pages, logs, or third-party APIs for context, attackers can inject carefully crafted adversarial prompts into these data streams. Unlike traditional prompt injection targeting data theft or prohibited content, termination poisoning solely aims to hijack the agent's control flow.

Table 4: LoopTrap attack strategy taxonomy [9].

Strategy	Mechanism
Progress bar manipulation	Forge incomplete progress indicators
System instruction cognitive bias	Exploit model's blind obedience to system-level commands
Dependency structure tampering	Claim unfinished hidden prerequisites
Fictitious scoring mechanism	Perpetually unsatisfied evaluation criteria

In extensive evaluations, attacked models (including GPT-5.5 and Claude Opus) enter a "self-doubt" logical dead loop, continuously calling verification tools without purpose. Total execution steps are amplified by 3.57× to as high as 25×, causing uncontrolled compute consumption and system-level denial of service[9].

5.2 Silent Failures

While termination poisoning is external, *silent failures* represent an internal entropy curse inherent to complex systems[10]. In production observations spanning hundreds of thousands of agent interactions, silent failure is defined as: task accuracy degradation and behavioral disorder occurring without adversarial input, resource exhaustion, or any individual software component raising an error.

Silent failures are extraordinarily stealthy. They manifest as accumulation of microscopic errors, with agent nodes confidently reporting "operation successful" even as their internal state has drifted from correctness[10]. Research identifies three specific manifestations[7]:

- **Channel Fracture**: inter-agent context injection reports success but critical state is lost.
- **Cognitive Framework Lag**: the model's initial adherence to response specifications and role constraints gradually erodes over long task horizons.
- **Data Consistency Decay**: biases accumulate progressively through complex data flow pipelines.

5.3 The Intelligence Entropy Principle

To formalize this inevitability, researchers have proposed the *Intelligence Entropy Principle*[8]:

$$S(t) = S_0 \cdot e^{\lambda t} \tag{6}$$

where $S(t)$ represents the system's disorder at interaction round t, and the constant λ is jointly determined by the model's intelligence density, the system's architectural topology complexity, and the task's inherent difficulty[10]. This exponential equation states a harsh physical reality: **autonomous agent systems relying purely on natural language for information transmission, lacking rigid structural constraints, will inevitably descend into chaos regardless of their initial perfection**, absent continuous external deterministic intervention and reset.

6 Toward Future Architecture: Multi-Agent Orchestration and Real-Time Entropy Evaluation

Facing the fundamental capacity limits of LLMs and the inexorable entropy growth, next-generation systems engineering is accelerating a paradigm shift from "granting unlimited

autonomy to a single model" toward "constructing tightly bounded, multi-center collaborative architectures."

6.1 Heterogeneous Multi-Agent Orchestration

Early homogeneous scaling strategies—deploying many identically configured large models in parallel to reduce error rates—exhibit severe diminishing returns[6]. Researchers have introduced the *effective channel number* metric, computed via eigenvalue entropy:

$$C_{\text{eff}} = \exp\left(-\sum_{i=1}^{N} \lambda_i \log \lambda_i\right) \tag{7}$$

where λ_i is the normalized eigenvalue spectrum distribution. This metric demonstrates that in systems lacking diversity, model-generated reasoning trajectories are highly redundant, failing to increase substantive effective channels[6]. Conversely, heterogeneous configurations—deploying broad-knowledge models for architectural planning, rigorous-logic models for core implementation, and low-cost lightweight models for high-frequency testing and verification—activate more effective reasoning channels with fewer total nodes, dramatically improving system-wide entropy reduction reliability and robustness.

6.2 Physical Integrity Gateway

On the engineering defense front, the Physical Integrity Gateway (PIG) and Agent Delivery Engineering (ADE) protocol are becoming infrastructure standards[10]. The core philosophy is to completely break blind trust in language models' subjective state reports. At every critical node—inter-agent task handoff, memory state injection, output persistence—the system introduces hard-assertion mechanisms based on hard-coded logic, not LLM judgment. This physical-level validation establishes an insurmountable moat around fragile language interactions, fundamentally containing the entropy growth constant λ and maintaining high agent survival rates over long-period asynchronous scheduling [8].

6.3 Meta-Repo Pattern

To mitigate the "cold-start high-entropy" problem at each session launch, project knowledge management is shifting toward the *Meta-Repo Pattern*[15, 17]. Rather than expecting agents to discover patterns in sprawling codebases via long context, developers establish rigorously structured workflow files: `SOUL.md` defining agent persona and principles, `AGENTS.md` providing project topology indices, and machine-readable build configurations like `repos.yaml`. This transforms implicit high-entropy team conventions into low-entropy constants that models can readily parse [14]. Such "document once, apply globally" infrastructure dramatically compresses the model's exploration space and computational cost before execution.

6.4 Real-Time Entropy Evaluation System (RTEES)

Based on our theoretical deconstruction and engineering analysis, we propose a novel system-level middleware to address the current architectural gap: the **Real-Time Entropy Evaluation System** (RTEES).

Current monitoring tools (e.g., `claude-hud`) are limited to passive display of physical resource metrics (token usage, latency). They completely lack dynamic awareness of the system's logical task state. Our proposed RTEES comprises three core components:

Real-Time Entropy Estimator. This module continuously samples the current dialog stack trajectory, variance across multiple consecutive tool call results, and the dispersion of the underlying LLM's output token probability distribution (logprobs). It computes the remaining task information uncertainty $\hat{H}(t)$ and risk index $\mathcal{R}(t)$:

$$\mathcal{R}(t) = \sigma\left(\alpha \cdot \hat{H}(t) + \beta \cdot \frac{d\hat{H}}{dt}\right) \tag{8}$$

where $\sigma(\cdot)$ is the Sigmoid function, α and β are tunable weights, and $d\hat{H}/dt$ is the entropy change rate.

Dynamic Routing and Strategy Recommendation Engine. When the engine detects that $\hat{H}(t)$ has flattened over multiple iterations without significant convergence—potentially indicating the model has hit the single-pass accuracy cliff or been trapped in a LoopTrap dead loop—it immediately halts the current model instance via a hardware-interrupt-level instruction. It then forcibly switches the entropy reduction path based on a statistical matrix of historical reduction efficiency: either escalating to a human developer for urgent clarification or transferring control to a higher-capacity arbitration model.

Objective Termination Criterion. The system redesigns the task exit mechanism. Under the new criterion, a legitimate task-completion signal is emitted only when:

$$\text{Stop} \iff \hat{H}(t) < H_{\text{threshold}} \ \wedge \ \text{PIG}_{\text{assert}}(\text{state}) = \text{True} \tag{9}$$

This mechanism permanently removes the LLM's absolute discretion to declare task completion based solely on subjective reasoning, providing the most robust objective guarantee for the agent loop's termination.

7 Conclusion

The deep deconstruction of Claude Code and similar frontier tools reveals a profound software engineering transformation underway. As LLM capabilities saturate, raw code generation is rapidly commoditizing toward an infrastructure utility akin to compute or electricity. The truly scarce and defensible value in this ecosystem is the engineering capacity to systematically transform chaotic, high-entropy, ambiguous real-world requirements into structured, low-entropy, machine-executable instruction sequences.

From the intersection of information theory and complex systems, current mainstream agent architectures have not yet solved the theoretical challenge of maintaining deterministic goals in unstructured high-entropy environments. As the accuracy cliff in single-pass reasoning and the inexorable Intelligence Entropy growth in long-running systems demonstrate, no matter how large the underlying model's parameter count, without strong physical fact anchors and rigorous engineering constraints, purely natural-language-controlled agents will inevitably descend into silent failure decay or adversarial dead loops.

We summarize five core contributions of this paper:

1. **Unified information-theoretic framework**: deconstructing LLM agent systems as multi-layer entropy reduction pipelines.

2. **Fano bound application**: mathematically proving the necessity of tool-call decomposition from first principles.

3. **Intelligence Entropy Principle**: formalizing long-run degradation as $S(t) = S_0 e^{\lambda t}$.

4. **Effective channel model**: quantifying heterogeneous multi-agent orchestration benefits via C_{eff}.

5. **RTEES architecture**: designing a complete middleware for real-time entropy monitoring, dynamic routing, and objective termination.

We foresee that the core competitiveness of next-generation AI systems will shift from single-model benchmark scores to the maturity of **multi-agent orchestration and control engineering**. Only by constructing heterogeneous multi-agent collaborative networks, deeply integrating structured communication standards like MCP, and embedding probabilistic reasoning kernels within a deterministic track of fact verification, real-time entropy monitoring, and physical integrity gates, can the research community and industry finally bridge the fragility divide and nurture trustable, industrially reliable autonomous development systems at scale.

References

[1] Onyx AI. Best LLMs in 2026: Rankings and benchmark comparison, 2026. URL https://onyx.app/insights/best-llms-2026.

[2] Vellum AI. LLM leaderboard 2026 — compare top AI models, 2026. URL https://www.vellum.ai/llm-leaderboard.

[3] Artificial Analysis. Terminal-Bench v2.1 benchmark leaderboard, 2026. URL https://artificialanalysis.ai/evaluations/terminalbench-v2-1.

[4] Anthropic. Model context protocol specification, 2025. URL https://modelcontextprotocol.io/specification/2025-11-25. Version 2025-11-25.

[5] Anonymous Authors. A fano-style accuracy upper bound for LLM single-pass reasoning in multi-hop QA. *arXiv preprint arXiv:2509.21199*, 2025. URL https://arxiv.org/abs/2509.21199.

[6] Anonymous Authors. Understanding agent scaling in LLM-based multi-agent systems via diversity. *arXiv preprint arXiv:2602.03794*, 2026. URL https://arxiv.org/abs/2602.03794.

[7] Anonymous Authors. Channel fracture: Architectural blind spots in scheduled cross-agent memory injection for multi-agent orchestration systems. *arXiv preprint arXiv:2606.04896*, 2026. URL https://arxiv.org/abs/2606.04896.

[8] Anonymous Authors. Intelligence entropy principle and the ADE stability engineering framework. *arXiv preprint*, 2026. URL https://www.researchgate.net/publication/407171586.

[9] Anonymous Authors. LoopTrap: Termination poisoning attacks on LLM agents. *arXiv preprint arXiv:2605.05846*, 2026. URL https://arxiv.org/abs/2605.05846.

[10] Anonymous Authors. Silent failure in LLM agent systems: The entropy principle and the inevitable disorder of autonomous agents. *arXiv preprint arXiv:2606.08162*, 2026. URL https://arxiv.org/abs/2606.08162.

[11] Encore Cloud. What is mcp (model context protocol)? a developer's guide, 2026. URL https://encore.dev/articles/what-is-mcp.

[12] CodingFleet. SWE-bench Pro leaderboard 2026: 30 AI models ranked, 2026. URL https://codingfleet.com/blog/swe-bench-pro-leaderboard-2026/.

[13] Scale Labs. AI model leaderboards & benchmarks, 2026. URL https://labs.scale.com/leaderboard.

[14] RinDig. Interpreted-context-methodology: Folder structure as agent architecture, 2026. URL https://github.com/RinDig/Interpreted-Context-Methdology.

[15] Seylox. In which We give Our AI agent a map (and it stops getting lost), 2026. URL https://seylox.github.io/2026/03/05/blog-agents-meta-repo-pattern.html.

[16] Iternal Technologies. Which LLM to choose in 2026? selection guide + benchmarks, 2026. URL https://iternal.ai/llm-selection-guide.

[17] WebbyWisp. How I structure My AI agent workspace (and why it matters), 2026. URL https://dev.to/webbywisp/how-i-structure-my-ai-agent-workspace-and-why-it-matters-j13.

[18] Wikipedia contributors. Model context protocol, 2026. URL https://en.wikipedia.org/wiki/Model_Context_Protocol. Accessed: 2026-06-30.

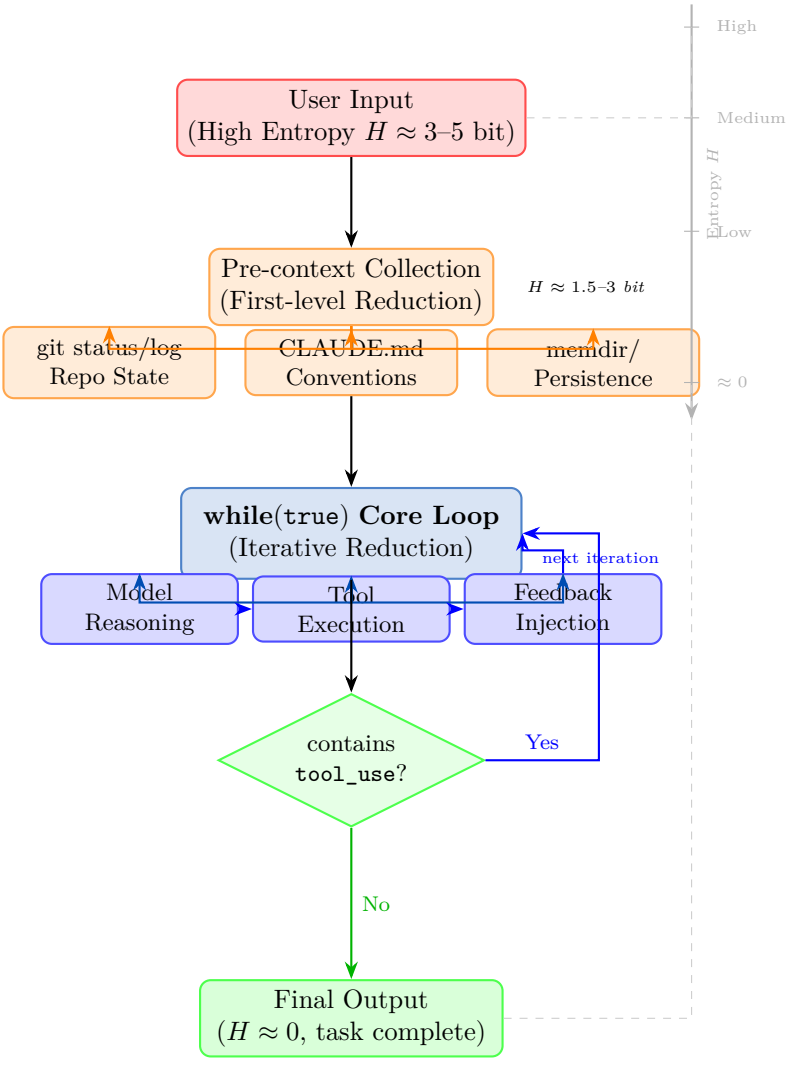

Figure 1: The Claude Code entropy reduction pipeline: from high-entropy user input to low-entropy output through iterative context collection and tool-augmented loops.

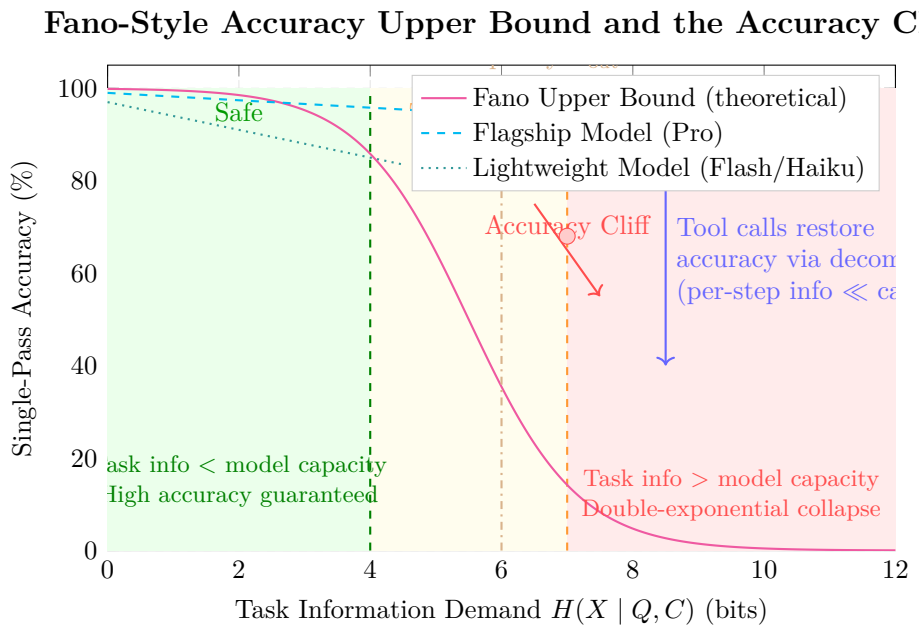

Figure 2: The Fano-style accuracy upper bound and accuracy cliff phenomenon. When task information demand exceeds model capacity, accuracy undergoes catastrophic collapse. Tool calling restores per-step accuracy through task decomposition.

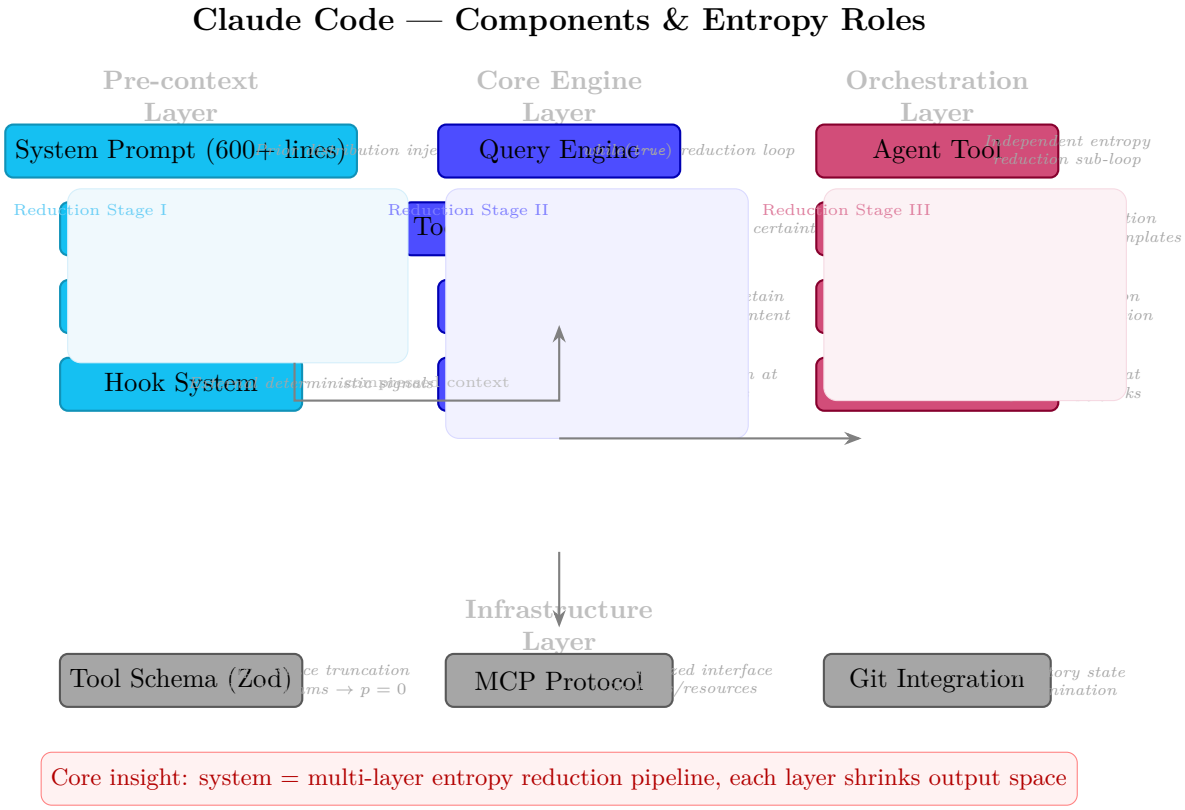

Figure 3: Claude Code system components mapped to their information-theoretic entropy roles.